

EXPERIMENT NO. 8

Implement custom middleware for user authorization in an Express app

Name: Rohit Pujari Roll No.: 28 Date: _____

Aim:

To secure “CampusLibrary”, a library REST API built with Express on port 3008, using only hand-written middleware: a request logger, a signed-token authentication middleware (HMAC-SHA256, without any authentication library), a login-required check, a role-based authorize(...roles) factory, an owner-or-staff check for loan records, a rate limiter on the login route, and central 404 and error handlers.

Theory:

In Express a middleware is a function (req, res, next). It can read or change the request, end the request by sending a response, or call next() to hand control to the next function. Middleware registered with app.use() runs for every request in the order it was added; middleware passed as extra arguments to a route runs only for that route, from left to right, before the final handler. An error handler is recognised by its four parameters (err, req, res, next).

Signed tokens instead of server-side sessions

After a successful login the server returns a token made of two parts: base64url(payload) + "." + base64url(HMAC-SHA256(payload, secret)). The payload contains the user id, the role and an expiry time 15 minutes ahead. The client sends it back in the header Authorization: Bearer <token>. The server recomputes the HMAC with its secret; if even one character of the payload was changed (for example the role “student” replaced by “admin”), the signature no longer matches and the request is rejected. This is the same idea as a JSON Web Token, written by hand.

- **Stateless:** the server keeps nothing per login, so any number of server instances can check a token. The drawback is that a token cannot be cancelled before it expires, which is why the lifetime is short.
- **Encoded, not encrypted:** anyone can read the payload, so it holds no password or secret data.
- **Password storage:** passwords are never stored; crypto.scryptSync() stores a salted hash.
- **Timing-safe comparison:** crypto.timingSafeEqual() compares signatures in constant time, so an attacker cannot guess a signature byte by byte from response times.

The middleware chain

Middleware	Registered	Responsibility	On failure
requestLogger	app.use (global)	Gives each request an id; logs method, URL, status, user and time on the finish event	-
express.json()	app.use (global)	Parses the JSON body into req.body	400 Invalid JSON body
authenticate	app.use (global)	If a Bearer token is present, verifies it and sets req.user; guests pass through	401 Invalid token
requireLogin	per route	req.user must exist	401 Login required
authorize(...roles)	per route	req.user.role must be in the list	403 Forbidden
selfOr(...roles)	/api/loans/:userId	Own user id, or a staff role	403 Forbidden
rateLimit(5, 60000)	/api/login	At most 5 login attempts per IP per minute	429 Too many
404 and error handler	last	Unknown route / unexpected exception	404 / 500

Authentication, authorization and status codes

Authentication answers “who are you?” (token check) and authorization answers “what may you do?” (role and ownership). authorize() and selfOr() are factory functions: calling authorize('librarian', 'admin') returns a new middleware that remembers the allowed roles through a closure. The API uses 401 when the caller is not logged in or the token is invalid, 403 when the caller is known but not allowed, 404 for a missing book or route and 429 for too many login attempts.

Route	Guest	Student	Librarian	Admin
GET /api/books	Yes	Yes	Yes	Yes
GET /api/me	401	Yes	Yes	Yes
GET /api/loans/:userId	401	Own only	Any user	Any user

POST /api/books	401	403	Yes	Yes
DELETE /api/books/:id	401	403	403	Yes

A test script (test.js) calls every route with fetch() as a guest, a student, the librarian and the admin, and also sends a wrong password and a tampered token. Its output is shown in Figure 8.1 and the matching server log in Figure 8.2.

Important Code:

Signing and verifying a token

```
const b64 = s => Buffer.from(s).toString('base64url');
function sign(user) {
  const payload = b64(JSON.stringify({ sub: user.id, role: user.role, exp:
Math.floor(Date.now() / 1000) + TOKEN_TTL }));
  const sig = crypto.createHmac('sha256', SECRET).update(payload).digest('base64url');
  return `${payload}.${sig}`;
}
function verify(token) {
  const [payload, sig] = String(token).split('.');
  if (!payload || !sig) throw new Error('malformed token');
  const expected = crypto.createHmac('sha256', SECRET).update(payload).digest();
  const given = Buffer.from(sig, 'base64url');
  if (given.length !== expected.length || !crypto.timingSafeEqual(given, expected)) throw new
Error('bad signature');
  const data = JSON.parse(Buffer.from(payload, 'base64url').toString());
  if (data.exp < Date.now() / 1000) throw new Error('token expired');
  return data;
}
```

Authentication and authorization middleware

```
function authenticate(req, res, next) { // who are you? (optional)
  const header = req.get('Authorization') || '';
  if (!header.startsWith('Bearer ')) return next(); // guest request
  try {
    const data = verify(header.slice(7));
    req.user = users.find(u => u.id === data.sub);
    if (!req.user) throw new Error('unknown user');
    next();
  } catch (err) {
    res.status(401).json({ error: `Invalid token: ${err.message}` });
  }
}

const requireLogin = (req, res, next) =>
  req.user ? next() : res.status(401).json({ error: 'Login required' });

const authorize = (...roles) => (req, res, next) =>
  roles.includes(req.user.role) ? next()
  : res.status(403).json({ error: `Role '${req.user.role}' cannot ${req.method} $
{req.baseUrl}${req.path}` });

// Owner of the resource, or one of the given staff roles.
const selfOr = (...roles) => (req, res, next) =>
  Number(req.params.userId) === req.user.id || roles.includes(req.user.role) ? next()
  : res.status(403).json({ error: 'You can only view your own loans' });
```

Routes - middleware composed per route

```
app.get('/api/books', (req, res) => res.json(books)); // public
app.get('/api/me', requireLogin, ({ user }, res) => res.json({ id: user.id, username:
user.username, role: user.role }));
app.get('/api/loans/:userId', requireLogin, selfOr('librarian', 'admin'),
  (req, res) => res.json(loans.filter(l => l.userId === Number(req.params.userId))));
app.post('/api/books', requireLogin, authorize('librarian', 'admin'), (req, res) => { ... })
app.delete('/api/books/:id', requireLogin, authorize('admin'), (req, res) => { ... })
```

Output:

```
rohit@laptop: ~/wp-practicals
rohit@laptop:~/wp-practicals$ node exp08/test.js
guest lists books      GET    /books      200    [{"id":101,"title":"Clean Code","copies":3},{ "id":102,"title
guest reads profile    GET    /me         401    {"error":"Login required"}
wrong password         POST   /login      401    {"error":"Wrong username or password"}
login rohit            POST   /login      200    {"token":"eyJzdWIiOiJlbnVjbGUiOi..."}
login mehta            POST   /login      200    {"token":"eyJzdWIiOiJlbnVjbGUiOi..."}
login admin            POST   /login      200    {"token":"eyJzdWIiOiJlbnVjbGUiOi..."}
rohit reads profile    GET    /me         200    {"id":1,"username":"rohit","role":"student"}
rohit own loans        GET    /loans/1    200    [{"userId":1,"bookId":101,"due":"2026-10-12"}]
rohit loans of priya   GET    /loans/2    403    {"error":"You can only view your own loans"}
librarian loans of priya GET    /loans/2    200    [{"userId":2,"bookId":102,"due":"2026-10-05"}]
rohit adds book        POST   /books      403    {"error":"Role 'student' cannot POST /api/books"}
librarian adds book    POST   /books      201    {"id":103,"title":"Computer Networks","copies":4}
librarian deletes book DELETE /books/101 403    {"error":"Role 'librarian' cannot DELETE /api/books/101"}
admin deletes book     DELETE /books/101 200    {"id":101,"title":"Clean Code","copies":3}
tampered token         GET    /me         401    {"error":"Invalid token: bad signature"}
```

Figure 8.1: Output of test.js - 200, 201, 401 and 403 responses for guest, student, librarian and admin requests

```
rohit@laptop: ~/wp-practicals
rohit@laptop:~/wp-practicals$ node exp08/server.js
CampusLibrary API on http://localhost:3008
[dcfed88b] GET    /api/books  -> 200  guest      5.5 ms
[0ac4072a] GET    /api/me     -> 401  guest      1.1 ms
[a3f341ce] POST   /api/login  -> 401  guest      62.9 ms
[b5833801] POST   /api/login  -> 200  guest      42.6 ms
[ed6f2da4] POST   /api/login  -> 200  guest      43.4 ms
[a909d2d8] POST   /api/login  -> 200  guest      41.4 ms
[06265b31] GET    /api/me     -> 200  rohit(student) 1.1 ms
[3f628216] GET    /api/loans/1 -> 200  rohit(student) 0.8 ms
[c2d5107f] GET    /api/loans/2 -> 403  rohit(student) 1.1 ms
[bec733c6] GET    /api/loans/2 -> 200  mehta(librarian) 0.6 ms
[fc258b0e] POST   /api/books  -> 403  rohit(student) 0.7 ms
[f9435d35] POST   /api/books  -> 201  mehta(librarian) 0.9 ms
[a80f0926] DELETE /api/books/101 -> 403  mehta(librarian) 0.7 ms
[85628663] DELETE /api/books/101 -> 200  admin(admin) 0.7 ms
[dc8b80e6] GET    /api/me     -> 401  guest      0.5 ms
```

Figure 8.2: Server console written by the requestLogger middleware (request id, status, user and role, time)

Conclusion:

The CampusLibrary API was protected with custom Express middleware only. A hand-written HMAC-signed token identified the user without server-side sessions, authorize() and selfOr() enforced role and ownership rules, and the logger, rate limiter and error handlers completed the chain. The test script confirmed the correct 200, 201, 401 and 403 responses, and a tampered token was rejected.